

STACK

Unit 5

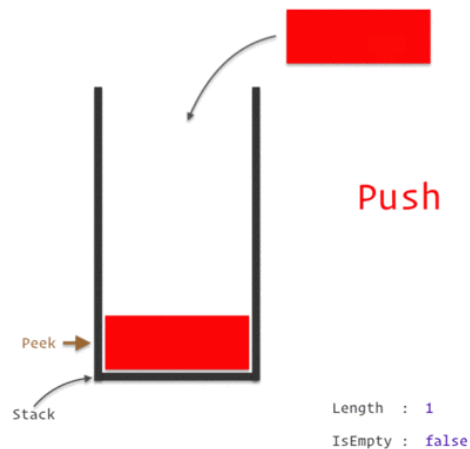
Presented By: Saroj Bhandari

Syllabus

- Introduction
- Stack as an ADT
- POP and PUSH Operation
- Stack Application: Evaluation of Infix, Postfix, and Prefix Expressions, Conversion of Expression.

Introduction to Stack

- **Stack** is a **linear data structure** that follows **LIFO (Last In First Out) Principle**, the last element inserted is the first to be popped out.
- It means both insertion and deletion operations happen at **one end only**.



Overview of Stack Operations

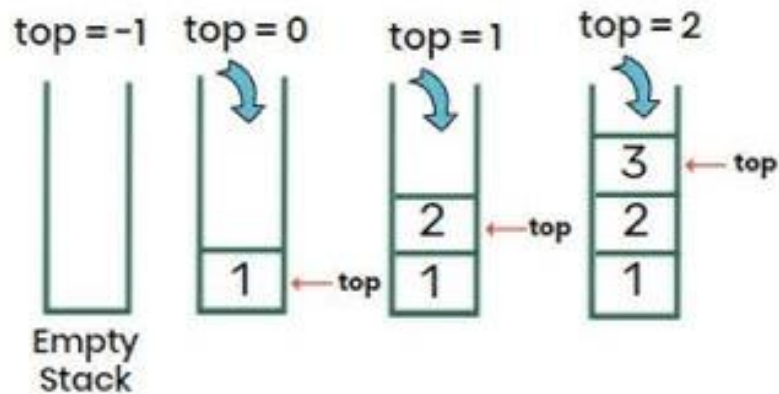


Stack Data Structure



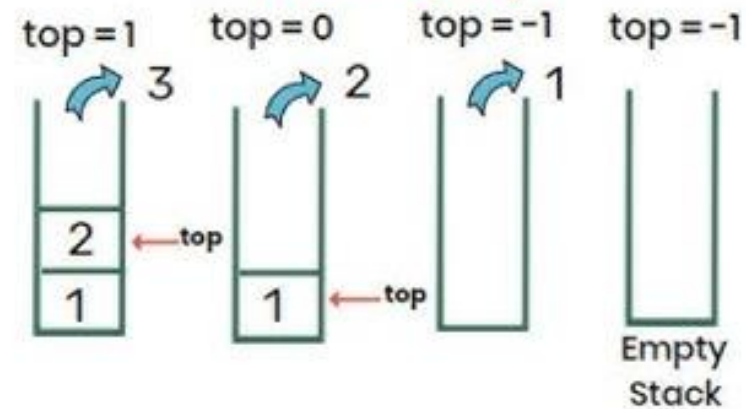
Push Operation

isFull



Pop Operation

isEmpty



Stack as an ADT

```
Stack {  
    // Push an element onto the stack  
    operation Push(x) -> void  
        ensures: top element is x  
  
    // Remove and return the top element from the stack  
    operation Pop() -> element  
        requires: Stack is not empty  
        ensures: top element is removed and returned  
  
    // Return the top element without removing it  
    operation Peek() -> element  
        requires: Stack is not empty  
        ensures: top element is returned without modification  
  
    // Check if the stack is empty  
    operation isEmpty() -> boolean  
        ensures: returns true if stack has no elements, false otherwise  
  
    // Return the size of the stack  
    operation Size() -> integer  
        ensures: returns the number of elements in the stack  
}
```

Basic Operations on Stack:

- In order to make manipulations in a stack, there are certain operations :
 - **push()** to insert an element into the stack
 - **pop()** to remove an element from the stack
 - **top()** Returns the top element of the stack.
 - **isEmpty()** returns true if stack is empty else false.
 - **isFull()** returns true if the stack is full else false.

Stack **push()** operation

- It stacks up a new item. A **stack overflow** circumstance is when the stack is completely full.

```
begin
  if stack is full
    return
  endif
  else
    increment top
    stack[top] assign value
  end else
end procedure
```

Stack **pop()** operation

- It takes something out of the stack.
- In the opposite sequence from which they were pushed, the things are popped.
- The condition is referred to as an **underflow** if the stack is empty.

```
begin
  if stack is empty
    return
  endif
  else
    store value of stack[top]
    decrement top
    return value
  end else
end procedure
```


Lab 1: Implementation of Stack in C

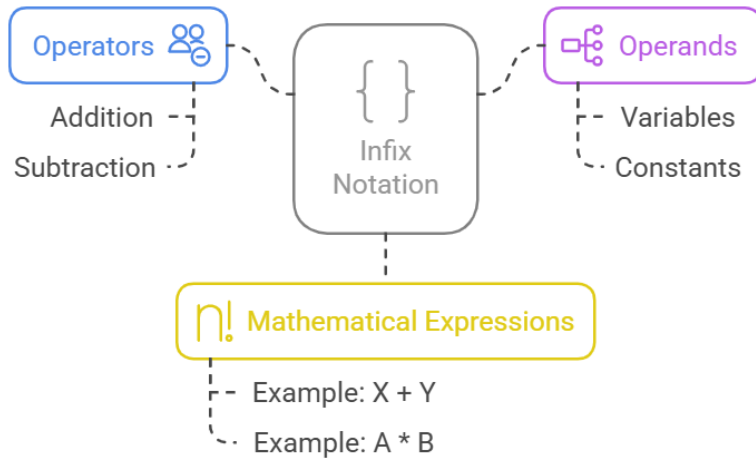
Objective:

- To implement stack push and pop operations using C programming Language

Infix, Prefix and Postfix Notations

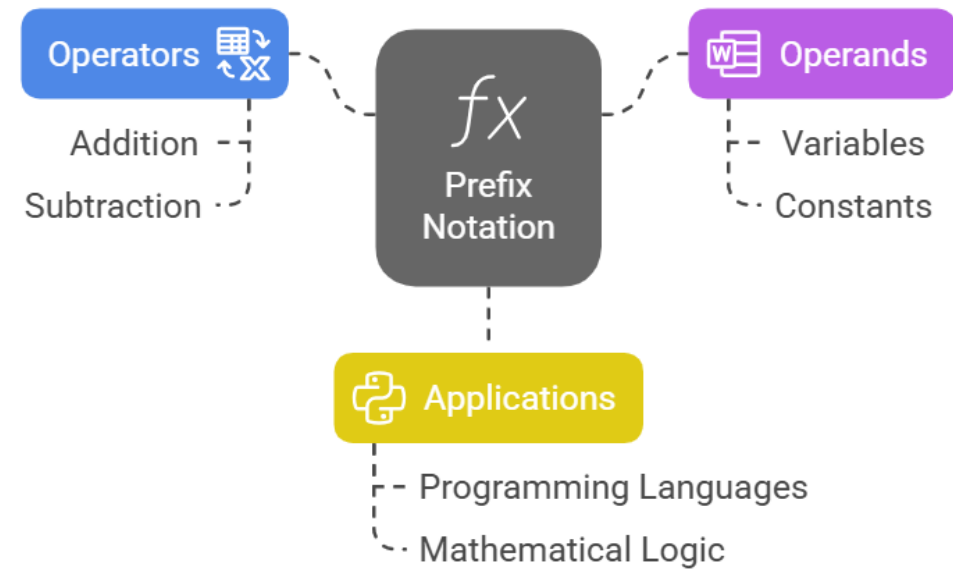
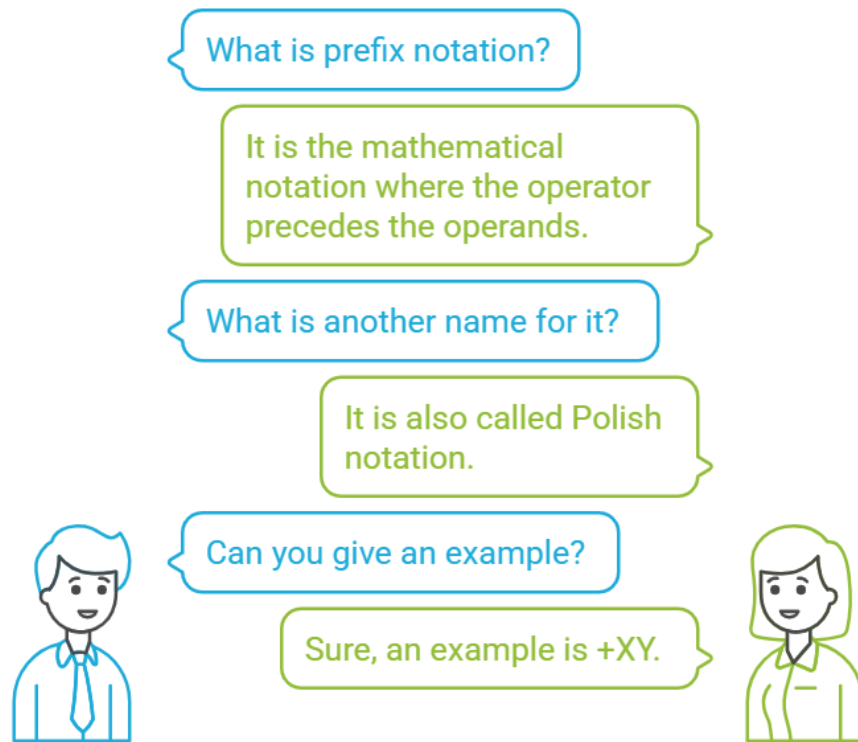
(1) Infix

- It is an ordinary mathematical notation of expression where an **operator is written in between operands**. The reason why this notation is called infix is the place of operator in the expression



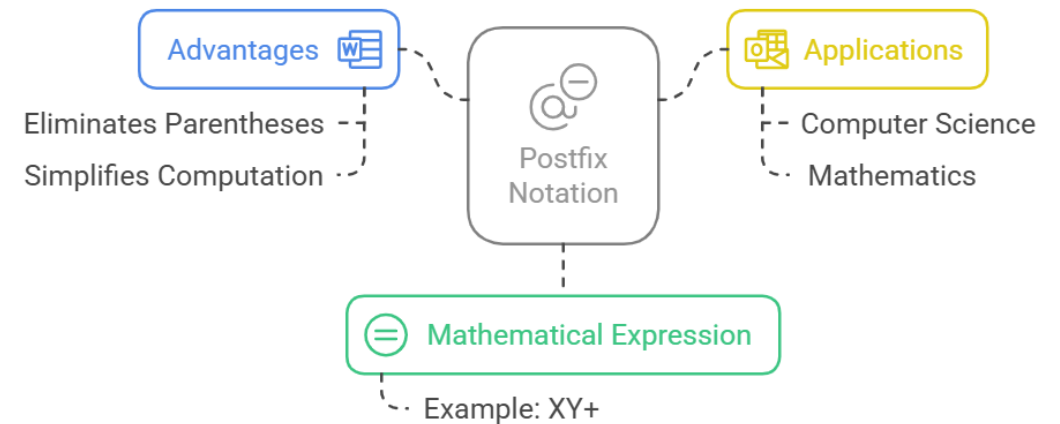
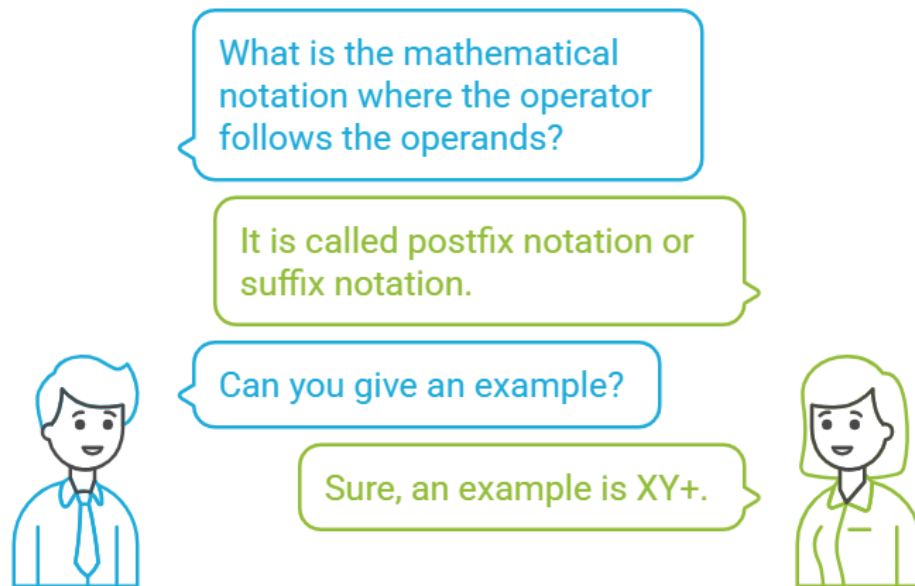
Infix, Prefix and Postfix Notations

(2) Prefix



Infix, Prefix and Postfix Notations

(3) Postfix

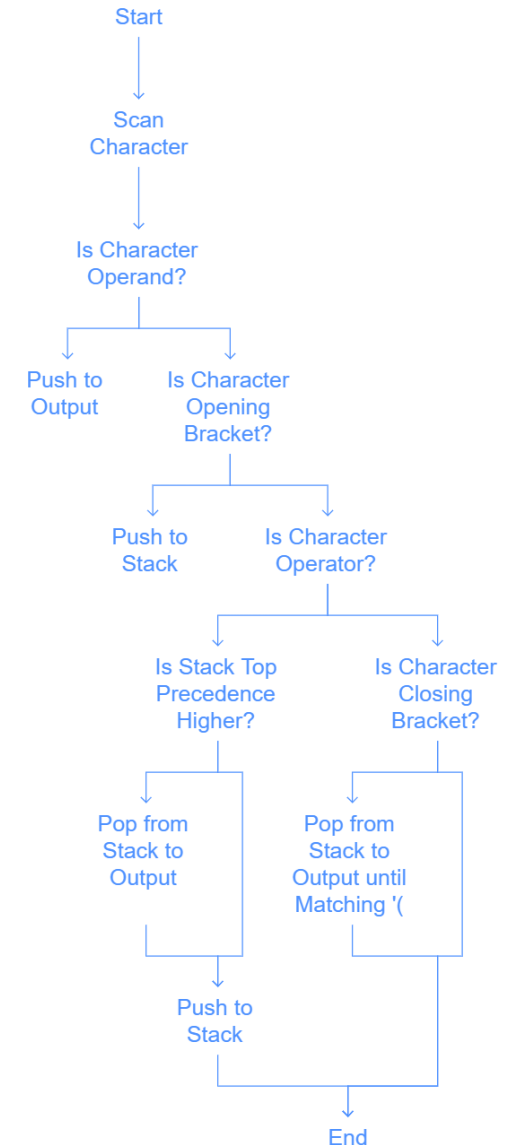


Operator Precedence

Operator	Precedence
Parentheses ()	Highest
Exponents ^	High
Multiplication *	Medium
Division /	Medium
Addition +	Low
Subtraction -	Low

Algorithm for Converting Infix to Postfix

- (1) Scan the given expression from left to right one character at a time
- (2) Repeat until there is no input character is left
 - (2.1) If operand, then push it to output(postfix) string
 - (2.2) If opening bracket ('('), then push it to stack
 - (2.3) If operator then
 - While precedence of top of stack character $\geq$ scanned character,
 - repeatedly pop from stack and push it to output string
 - Push it to stack
 - (2.4) If closing bracket (')') is found,
 - repeatedly pop from stack and push it to output string until matching '(' is found and cancel both '(' and ')'
- (3) Pop from stack and append to output string until stack is empty
- (4) The required **postfix expression is given by the output string**



Confusing ?

Lets look a Example

- Convert the following infix expression to postfix:

$A * B + C + D + E / F / (G + H)$

AB\$C*D+EF/GH+ / +

SN	Scanned	Postfix String	Stack

Let's Verify It ?

SNo.	Scanned	Postfix String	Stack
1	A	A	-----
2	\$	A	\$
3	B	AB	\$
4	*	AB\$	*
5	C	AB\$C	*
6	+	AB\$C*	+
7	D	AB\$C*D	+
8	+	AB\$C*D+	+
9	E	AB\$C*D+E	+
10	/	AB\$C*D+E	+ /
11	F	AB\$C*D+EF	+ /
12	/	AB\$C*D+EF/	+ /
13	(	AB\$C*D+EF/	+ / (
14	G	AB\$C*D+EF/G	+ / (
15	+	AB\$C*D+EF/G	+ / (+
16	H	AB\$C*D+EF/GH	+ / (+
17	)	AB\$C*D+EF/GH+	+ /
18	-----	AB\$C*D+EF/GH+/+	

Practice Questions

Convert into postfix:

(1) $A + (B * C - (D / E \$ F) * G) * H$

(2) $((A - (B + C)) * D) \$ (E + F)$

Practice Questions

- Ans1: A B C * D E F \$ / G * - H * +
- Ans2: A B C + - D * E F + \$

SN	Scanned	Postfix String	Stack

Algorithm for **Converting Infix to Prefix**

- (1) Scan the given expression from **right to left** one character at a time
- (2) **Repeat until there is no input character is left**
 - (2.1) If **operand**, then push it to output(prefix) string
 - (2.2) If **closing bracket (')')**, then push it to stack
 - (2.3) If **operator** then
 - While precedence of top of stack character > scanned character, repeatedly pop from stack and push it to output string
 - Push it to stack
 - (2.4) If opening bracket '(' is found, repeatedly pop from stack and push it to output string until matching ')' is found and cancel both '(' and ')'
- (3) **Pop from stack and append to output string until stack is empty**
- (4) The required prefix expression is given by **printing the output string in reverse order**

Example:

- **Convert to prefix:**

(1) $((A - (B + C)) * D) \$ (E + F)$

(2) $(A + B) * (C \$ (D - E) + F) - G$

Let's Explore one of the Examples:

SN	Scanned	Postfix String	Stack

Practice Questions

- Ans1: \$ * - A + B C D + E F
- Ans2:- * + A B + \$ C - D E F G

Algorithm for Evaluating Postfix Expression

- Let '**E**' be an arithmetic expression written in postfix notation, '**Symb**' be scanned character, '**op1**' and '**op2**' be operands 1 and 2 respectively, '**ValStk**' be the stack containing values of the operand
- (1) Scan the given expression from left to right one character at a time into **Symb**
 - (2) Repeat steps 3 and 4 until no input character is remaining
 - (3) If **Symb** is an operand, read its value and push it to **ValStk**
 - (4) If **Symb** is an operator then
 - (4.1) Pop and store it in **op2**
 - (4.2) Pop and store it in **op1**
 - (4.3) Get the result as "**op1 operator op2**" and push the result to **ValStk**
 - (5) Pop and print the required result from **ValStk**

Example

- Evaluate the following postfix expression:

$ABC + * CBA - + *$ with $A = 1, B = 2, C = 3$

Example

ABC + * CBA - + * with A = 1, B = 2, C = 3

SNo	Symb	Value	Op1	Op2	ValStk

Let's Check it :

SNo.	Symb	Value	op1	op2	ValStk
1	A	1	-----	-----	1
2	B	2	-----	-----	1 2
3	C	3	-----	-----	1 2 3
4	+	-----	2	3	1 5
5	*	-----	1	5	5
6	C	3	1	5	5 3
7	B	2	1	5	5 3 2
8	A	1	1	5	5 3 2 1
9	-	-----	2	1	5 3 1
10	+	-----	3	1	5 4
11	*	-----	5	4	20

Practice Questions

- Evaluate the following postfix expressions:

(1) $A B + C * D E - - F G + \$$ for $A = 1, B = 2, C = 3, D = 4, E = 3, F = 2, G = 1$

(2) $6 2 3 + - 3 8 2 / + * 2 \$ 3 +$

Practice Questions

- Ans1: 512
- Ans2: 52

Algorithm for Evaluating Prefix Expression

- (1) Scan the given expression from **right to left** one character at a time into **Symb**
- (2) Repeat steps 3 and 4 until no input character is remaining
- (3) If Symb is an operand, read its value and push it to ValStk
- (4) If Symb is an operator then
 - (4.1) Pop and store it in op1
 - (4.2) Pop and store it in op2
 - (4.3) Get the result as “op1 operator op2” and push the result to ValStk
- (5) Pop and print the required result from ValStk

Practice Question

- Evaluate the following prefix expression:

$\$ * - A + B C D + E F$

for $A = 6, B = 1, C = 2, D = 3, E = 2, F = 1$

Ans: 729

Multiple Stack

- ❑ A **"multiple stack"** refers to the concept of having more than one stack data structure within a single program or system.
- ❑ This can be implemented using a single array or separate data structures for each stack.
- ❑ It allows for efficient management of multiple independent "stacks" of data, like in a program with multiple undo/redo functionalities or in a system with separate stacks for different data domains.
- ❑ **Example:**
 - ❑ Consider a website with different sections like a blog, a documentation section, and an e-commerce section. Each section can have its own stack for managing content or versions.



Thank you !!!